Reporte de Vulnerabilidades — Práctica 1 UC3

> **Curso:** Ciberseguridad Aplicada · **Unidad de Competencia 3** · **ITSON Práctica:** P1 — Seguridad en APIs (BOLA · Mass Assignment · Validación estricta) · **Equipo:** Equipo Hector — Hector Andres Lagarda Alcantar (204733) · **Fecha:** 11/11/2026

Resumen general

Métrica	Valor
Total de hallazgos	5
Críticos	3
Altos	2
Medios	0
Bajos	0
Herramientas utilizadas	Postman (colección de explotación), Maven/NetBeans (ejecución), H2 (base en memoria)

Nota de entorno: el puerto 8080 estaba ocupado por otro servicio (GeoServer), por lo que la API se ejecutó en `http://localhost:8081` (alternativa indicada en la guía). Además, en ambos `data.sql` se retiraron los IDs explícitos de los INSERT porque H2 2.2.224 no avanza la secuencia IDENTITY al sembrar IDs manuales (los POST chocaban con la clave primaria). No hay cambio funcional: los datos sembrados conservan los mismos identificadores (Ana=1, Beto=2, pedidos 1–3).

Ficha de hallazgo

ID del hallazgo: P1-001

Campo	Contenido
Título	BOLA de lectura en `GET /api/pedidos/{id}`
Severidad	Crítica (acceso a datos de otros usuarios sin autorización con solo cambiar el identificador en la URL)
Referencia OWASP	API1:2023 — Broken Object Level Authorization
Endpoint / componente	`GET /api/pedidos/{id}` · clase `PedidoController.java`
Descripción	El endpoint devuelve el pedido solicitado por su identificador sin comparar el dueño del recurso contra el usuario autenticado (encabezado `X-User-Id`, que la versión vulnerable ignora por completo).
Evidencia	`evidencias/explotacion/02_bola_lectura_pedido_ajeno.png` — petición `GET /api/pedidos/3` con `X-User-Id: 1` (Ana) → **200 OK** con el pedido de Beto (Monitor 24 pulgadas) y sus datos personales.
Impacto	Exposición de datos de terceros (privacidad); permite enumerar pedidos ajenos cambiando el id (1, 2, 3…); base para ataques de modificación/borrado de datos ajenos.
Facilidad de explotación	Alta: basta cambiar el identificador en la URL; no se requiere autenticación real.
Recomendación (OWASP)	Verificar en el servidor la propiedad del objeto en cada acceso (`pedido.usuario.id == usuarioAutenticado.id`) y responder **403** cuando el recurso pertenezca a otro usuario; **401** si no hay identidad.

ID del hallazgo: P1-002

Campo	Contenido
Título	BOLA de escritura en `POST /api/pedidos` (dueño del pedido definido por el cliente)
Severidad	Crítica (modificación de datos a nombre de otros usuarios)
Referencia OWASP	API1:2023 — Broken Object Level Authorization
Endpoint / componente	`POST /api/pedidos` · clases `PedidoController.java` / `PedidoService.java`
Descripción	El pedido se enlaza directamente a la entidad `Pedido`; el campo `usuario` del cuerpo JSON decide quién es el dueño del nuevo pedido. El atacante puede crear pedidos a nombre de cualquier usuario.
Evidencia	`evidencias/explotacion/04_bola_escritura_pedido_falso.png` — `POST /api/pedidos` con `X-User-Id: 1` (Ana) y cuerpo `{"descripcion":"Pedido inyectado a nombre de Beto","monto":99999.00,"fecha":"2026-11-11","usuario":{"id":2}}` → **200 OK**, pedido creado a nombre de Beto (usuario id 2).
Impacto	Suplantación en la creación de recursos; fraude (montos/descripciones falsas atribuidas a terceros); contaminación de la información de otros usuarios.
Facilidad de explotación	Alta: solo se envía el id del usuario víctima en el JSON.
Recomendación (OWASP)	El dueño debe resolverse SIEMPRE de la sesión autenticada (aquí, `X-User-Id`), nunca del cuerpo; usar un DTO de entrada sin el campo `usuario`.

ID del hallazgo: P1-003

Campo	Contenido
Título	Mass Assignment (BOPLA): auto-registro con rol `ADMIN` en `POST /api/usuarios`
Severidad	Crítica (escalada de privilegios: un usuario cualquiera se crea como administrador)
Referencia OWASP	API3:2023 — Broken Object Property Level Authorization (Mass Assignment)
Endpoint / componente	`POST /api/usuarios` · clase `UsuarioController.java` (binding directo de la entidad `Usuario`)
Descripción	El controlador deserializa directamente la entidad `Usuario`; Jackson copia todo campo del JSON que coincida por nombre. El cliente envía `"rol":"ADMIN"` (e incluso `"id":99`) y el servidor lo persiste sin filtrar.
Evidencia	`evidencias/explotacion/05_mass_assignment_rol_admin.png` — `POST /api/usuarios` con `{"nombre":"Eva Atacante","email":"eva@atacante.mx","password":"clave12345","rol":"ADMIN"}` → **201 Created** con `"rol":"ADMIN"`; y `evidencias/explotacion/06_verificar_rol_admin.png` — `GET /api/usuarios/3` confirma el rol ADMIN persistido.
Impacto	Escalada de privilegios completa: el atacante se convierte en administrador del sistema y puede operar sobre datos de todos los usuarios.
Facilidad de explotación	Alta: una sola petición POST, sin autenticación previa.
Recomendación (OWASP)	Usar un DTO de entrada sin propiedades sensibles (`rol`, `id`); asignar el rol en el servidor (siempre `USER`); rechazar campos desconocidos (esquema estricto).

ID del hallazgo: P1-004

Campo	Contenido
Título	Exposición de datos sensibles: contraseñas en las respuestas JSON
Severidad	Alta (exposición masiva de credenciales de todos los usuarios)
Referencia OWASP	API3:2023 (propiedades no autorizadas) / API8:2023 (falta de minimización de datos)
Endpoint / componente	`GET /api/usuarios` , `GET /api/usuarios/{id}` , `GET /api/pedidos*` (entidades serializadas completas, sin `@JsonIgnore`)
Descripción	Los controladores devuelven las entidades JPA completas; el campo `password` no está excluido de la serialización y viaja en cada respuesta, incluso en los pedidos (que embeben al usuario dueño).
Evidencia	`evidencias/explotacion/00_recon_listar_usuarios.png` — `GET /api/usuarios` muestra `"password":"ana12345"` y `"password":"beto12345"` ; `evidencias/explotacion/03_bola_lectura_usuario_ajeno.png` — `GET /api/usuarios/2` muestra la contraseña de Beto; `evidencias/explotacion/04_bola_escritura_pedido_falso.png` — la respuesta del POST incluye la contraseña de Beto.
Impacto	Robo masivo de credenciales con una sola petición; compromiso de cuentas (los usuarios suelen reutilizar contraseñas).
Facilidad de explotación	Trivial: cualquier GET de la API expone el dato.
Recomendación (OWASP)	`@JsonIgnore` sobre `password` y DTOs de salida que solo expongan los campos necesarios (minimización de datos).

ID del hallazgo: P1-005

Campo	Contenido
Título	Validación inadecuada de esquemas de entrada (montos extremos y campos sin validar)
Severidad	Alta (permite valores corruptos/no razonables en los datos)
Referencia OWASP	API8:2023 — Security Misconfiguration (falta `spring-boot-starter-validation`)
Endpoint / componente	`POST /api/pedidos` y `POST /api/usuarios` (sin `@Valid` , sin restricciones Bean Validation)
Descripción	Los cuerpos se aceptan sin validar: no hay restricciones de monto (`@Positive` , `@DecimalMax`), longitud de descripción, formato de email ni fortaleza de contraseña; la dependencia de validación está ausente del `pom.xml` .
Evidencia	`evidencias/explotacion/04_bola_escritura_pedido_falso.png` — el pedido con `"monto": 99999.00` se aceptó sin reclamar (200 OK), y el usuario de la evidencia 05 se registró con una contraseña trivial (`clave12345`) sin objeción.
Impacto	Datos corruptos (montos negativos o absurdos), cuentas con credenciales débiles, vectores para fraude y errores en cascada en otros módulos.
Facilidad de explotación	Alta: no se requiere ningún esfuerzo adicional, la API acepta todo.
Recomendación (OWASP)	Activar `spring-boot-starter-validation` con `@Valid` + restricciones (`@NotBlank` , `@Email` , `@Size` , `@Positive` , `@DecimalMax`) y manejo centralizado de errores 400 por campo.

Criterios de severidad (guía de uso)

Severidad	Cuándo usarla
Crítica	Acceso/modificación de datos de otros usuarios sin autorización (BOLA, mass assignment a rol admin), ejecución de código, o exposición masiva de credenciales.
Alta	Exposición de datos sensibles, validación inadecuada que permite valores corruptos, falta de rate limit en operación costosa.
Media	Información técnica filtrada (stacktrace, cabeceras), secretos en repositorio con acceso restringido.
Baja	Falta de cabeceras de seguridad, mensajes de error poco informativos, buenas prácticas ausentes sin impacto directo.

Checklist de cierre

- [x] Todos los hallazgos tienen evidencia capturada (petición y respuesta completas).
- [x] Cada hallazgo tiene severidad justificada y referencia OWASP.
- [x] Los IDs de hallazgo coinciden con los del reporte de correcciones.
- [x] El documento se generó SIN texto de ejemplo de la plantilla.